PROJECT REVIEW III – FINAL REVIEW

Progress, Individual Contribution and Project Outcome Report

KELVIN: AN AI-ASSISTED DOMAIN-SPECIFIC LANGUAGE COMPILER FOR IoT AUTOMATION

BCSE307P – Compiler Design Lab

Assessment 10

Submitted by

Name	Register Number
R. Sanjay	24BCI0181
Settipalli Tejovardhan	24BCI0196
Ashutosh	23BCI0117

Under the guidance of

Prof. Sathya K

VELLORE INSTITUTE OF TECHNOLOGY

Vellore – 632014, Tamil Nadu, India

October 2026

SUMMARY

This document accompanies our final review. The project, Kelvin, is complete: every compiler phase, the AI assistant with its verification gate, the four code generators, the web studio and the digital twin all work, and the full test suite of 145 tests passes. Section 1 records the completion status module by module. Section 2 describes what each of the three of us built and the topics each person can explain during the viva. Section 3 covers the project outcome required for this review: the target conference, the manuscript prepared in the IEEE conference template (submitted along with this report), the measurable results from a new experiment we ran for the paper, and how our work compares with recent research. Section 4 explains how we kept the written material original.

TABLE OF CONTENTS

SUMMARY i
1. PROJECT COMPLETION STATUS 1
2. INDIVIDUAL CONTRIBUTION 2
2.1 R. Sanjay (24BCI0181) 2
2.2 Settipalli Tejovardhan (24BCI0196) 2
2.3 Ashutosh (23BCI0117) 3
3. PROJECT OUTCOME 4
3.1 Target Venue 4
3.2 Manuscript 4
3.3 Measurable Outcomes 4
3.4 Novelty Compared with Recent Research 5
4. INTEGRITY OF THE SUBMITTED WORK 6

1. PROJECT COMPLETION STATUS

All planned work has been implemented and tested. Table 1 lists each module with the evidence that it works. Everything can be demonstrated live from one laptop with `make run`, and offline if no AI service is reachable.

Table 1: Completion status of every module

Module	Status	Evidence
Lexical analysis (DFA scanner)	Complete	Token table, DFA drawn in the studio, lexical error tests
Syntax analysis (LL(1) parser)	Complete	Grammar verified by the compiler: 61 productions, 0 conflicts
Abstract syntax tree	Complete	Tree view and graph in the studio
Semantic and rule analysis	Complete	Type errors, suggestions, conflict warnings with example inputs
Intermediate code (TAC)	Complete	Original and optimized listings
CFG, dataflow and optimizer	Complete	8 passes; example 07 shrinks from 21 to 10 instructions
Code generation (4 targets)	Complete	C, Arduino, Python and JavaScript agree in tests
AI assistant and verification gate	Complete	96.7% correct on our 30-rule benchmark
Web studio, X-ray, digital twin	Complete	Live demo; phone remote through QR code
Testing	Complete	145 automated tests, all passing
Manuscript (IEEE template)	Complete	Submitted with this report

2. INDIVIDUAL CONTRIBUTION

We divided the work by compiler module so that each of us owned a part of the pipeline from design to testing. We reviewed each other's code and all three of us can run the full demo, but the areas below are where each person did most of the work and can answer detailed questions.

Table 2: Division of work

Member	Register No.	Main contribution
R. Sanjay	24BCI0181	Lexer, parser, syntax tree, grammar analysis, web studio
Settipalli Tejovardhan	24BCI0196	Semantic and rule analysis, intermediate code, CFG optimizer, code generation
Ashutosh	23BCI0117	AI assistant and verification gate, test suite, benchmark, documentation

2.1 R. Sanjay (24BCI0181)

Sanjay built the front of the compiler. He wrote the scanner as a hand-coded finite automaton with longest-match tokenisation and position tracking, and added the lexical error messages with their quick fixes. He then wrote the recursive-descent parser, designed the grammar so that it is LL(1), and implemented panic-mode recovery so that several syntax errors can be reported at once. He also wrote the grammar-analysis module that computes FIRST and FOLLOW sets and the LL(1) table from the grammar stored inside the compiler. On the interface side he built the studio's editor, the syntax-tree graph and the Compiler X-ray that replays the scanner and parser traces.

Can explain: DFA construction and maximal munch; FIRST/FOLLOW computation and why the grammar has no LL(1) conflicts; recursive descent and operator precedence; error recovery; how parser traces drive the X-ray.

2.2 Settipalli Tejovardhan (24BCI0196)

Tejovardhan built the middle and back end. He implemented the symbol table and type checking, the *did-you-mean* suggestions, and the rule analysis that converts conditions to disjunctive normal form and uses intervals and value sets to find impossible, always-true, unreachable and conflicting rules. He designed the three-address intermediate code, split it into basic blocks, built the control-flow graph, and wrote the liveness and available-expressions analyses used by the eight optimization passes. He also wrote the structured emitter and the C, Arduino, Python and JavaScript back ends with their runtimes.

Can explain: type rules; DNF and the satisfiability test used for conflicts; three-address code; basic blocks and CFG construction; dataflow equations and fixed-point iteration; each optimization pass; recovering if–else structure during code generation.

2.3 Ashutosh (23BCI0117)

Ashutosh built the AI side and the verification around it. He integrated the three free model providers (Ollama, Groq and Gemini), wrote the prompts, and implemented the verification gate: compiler-guided repair, the meaning check that catches dropped numbers and unused sensors, and the fallback to the rule-based converter, which he also wrote. He set up the test suite and the cross-target equivalence tests, designed and ran the 30-rule benchmark used in the manuscript, measured the compiler's timing, and prepared the reports and the manuscript.

Can explain: how the gate decides to accept, repair or reject an answer; why a meaning check was needed; how behavioural correctness is measured in the benchmark; the test strategy; the experimental results and their limitations.

3. PROJECT OUTCOME

3.1 Target Venue

We prepared the manuscript for an IEEE conference using the official two-column IEEE conference template. Our first choice is the **IEEE International Conference on Computing, Communication and Networking Technologies (ICCCNT)**, which accepts work on IoT systems and applied software and publishes accepted papers through IEEE Xplore. Our second choice is **IEEE INDICON**, the annual conference of the IEEE India Council. Both are held every year in India and accept papers in this format. Before submitting, we will check the current call for papers for the exact deadline, page limit and author guidelines, and adjust the manuscript if needed.

3.2 Manuscript

Table 3: Manuscript details

Item	Details
Title	Kelvin: Compiler-Verified Natural-Language Programming of IoT Automation Rules
Authors	R. Sanjay, Settipalli Tejovardhan, Ashutosh, Sathya K
Format	IEEE conference template, two columns, US Letter
Length	5 pages, 4 figures, 3 tables, 20 references
Sections	Introduction, Related Work, Rule Language, Compiler Design, Verification Gate, Implementation, Evaluation, Discussion and Novelty, Conclusion
Files	Review_III_Manuscript.docx and Review_III_Manuscript.pdf

3.3 Measurable Outcomes

For the paper we ran a new experiment. We wrote 30 English automation rules and, for each, a reference program. Every answer was compiled to Python and run on up to 60 test inputs; it counted as correct only if it switched every device exactly as the reference did. We compared the rule-based converter, the raw output of the local Qwen3 8B model, the model with compiler-error repair, and the complete Kelvin gate.

Table 4: Results of the English-to-DSL experiment

Pipeline	Compiles	Behaviourally correct
Rule-based converter only	83.3%	83.3%
LLM output, unchecked	96.7%	90.0%
LLM with compiler-error repair	100.0%	93.3%
Kelvin verification gate	**100.0%**	**96.7%**

Other measured results reported in the manuscript:

- The gate raised accuracy by 6.7 percentage points over unchecked model output while adding only about 0.1 s to the median conversion time.
- The compiler finishes all seven phases in under 0.6 ms on each of the 13 example programs.
- The optimizer removes 52% of the instructions in the optimization example.
- All 145 regression tests pass, including tests that the C, Python and JavaScript outputs behave identically.

3.4 Novelty Compared with Recent Research

Table I of the manuscript compares Kelvin with eight systems published between 2018 and 2024. Safety analysers such as Soteria (2018), AutoTap (2019) and HAWatcher (2021) study existing smart-home programs but do not take English input. Recent work on checking model output, such as LEVER (2023) and Self-Debugging (2024), relies on executing the generated code, while Synchromesh (2022) and grammar-constrained decoding (2023) only guarantee correct syntax. Sasha (2024) applies language models to smart homes without checking the result against a language definition.

Kelvin differs in three ways. First, a complete domain compiler, not just a syntax check or a test run, decides whether a model's answer is accepted. Second, the compiler finds conflicting and unreachable rules at compile time and gives an example input for each. Third, one verified program is compiled to code for microcontrollers as well as for Python and JavaScript. The experiment shows that this design produces measurably more correct rules than using the model alone.

4. INTEGRITY OF THE SUBMITTED WORK

The manuscript and this report were written by our team and describe our own system. The results come from experiments we ran on our own code, and the scripts used to produce them are kept in the project so that they can be repeated. Every outside idea is cited, and the manuscript was written separately from our Review II report rather than copied from it, so that the two documents do not overlap.

The review guidelines require a Turnitin similarity of at most 10% and AI-generated content of at most 20%. We will run both documents through Turnitin before the final submission and revise any passage that is flagged.